

Parser

terça-feira, 22 de julho de 2025 07:56

↳ Representing the code

↳ tree that represents the grammatical structure

↳ Context free grammar (CFG)

↳ Limited of create infinite rules → create just what we can for now CLASS

Rules → Productions

↳ each production has a head & Body

Describes what it generates

Terminals

↳ tokens meaning end-points

↳ nothing derives from them → produce a symbol

In each step of the game, you pick a

rule and follow what it tells you to do. Most of the lingo around formal grammars comes from playing them in this direction. **Rules** are called **productions** because they produce strings in the grammar.

Each **production** in a context-free grammar has a **head**—its name—and a **body** which describes what it generates. In its pure form, the body is simply a list of **symbols**. Symbols come in two delectable flavors:

A **terminal** is a letter from the grammar's alphabet. You can think of it like a literal value. In the syntactic grammar we're defining, the terminals are individual lexemes—tokens coming from the scanner like if or 1234.

These are called "terminals", in the sense of an "end point" because they don't lead to any further "moves" in the game. You simply produce that one symbol.

A **nonterminal** is a named reference to another rule in the grammar. It means "play that rule and insert whatever it produces here". In this way, the grammar composes.

```
breakfast → protein "with" breakfast "on the side" ;
breakfast → protein ;
breakfast → bread ;
protein → crispiness "crispy" "bacon" ;
protein → "sausage" ;
protein → cooked "eggs" ;
crispiness → "really" ;
crispiness → "really" crispiness ;
cooked → "scrambled" ;
cooked → "poached" ;
cooked → "fried" ;
bread → "toast" ;
bread → "biscuits" ;
bread → "English muffin" ;
```

I tried to come up with something clean. Each rule is a name, followed by an arrow (→), followed by a sequence of symbols, and finally ending with a semicolon (;). Terminals are quoted strings, and nonterminals are lowercase words.

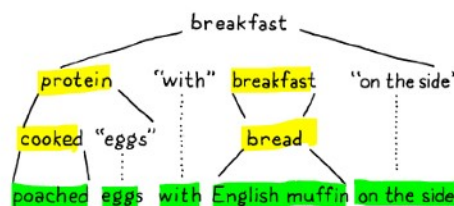
→ Expand the non-terminal w/ the possibilities to create a new sentence (expression w/ variables)

↳ There is no context in the code

→ Every non-terminal can be expanded until it contains only terminals

With that, every nonterminal in the string has been expanded until it finally contains only terminals and we're left with:

→ Means it's possible to have short number of grammars & encode a large set of strings



1/ SYMBOLS → Represent entire tokens

SYMBOLS → Represent entire tokens
 Rules →
 Productions →

→ I may need to declare this grammar rule for each ^{type}

EXPRESSIONS

- Literals – Numbers, strings, Booleans, and nil.
- Unary expressions – A prefix ! to perform a logical not, and - to negate a number.
- Binary expressions – The infix arithmetic (+, -, *, /) and logic (==, !=, <, <=, >, >=) operators we know and love.
- Parentheses – A pair of (and) wrapped around an expression.

In addition to quoted strings for **terminals that match exact lexemes**, we CAPITALIZE terminals that are a single lexeme whose text representation may vary. NUMBER is any number literal, and STRING is any string literal. Later, we'll do the same for IDENTIFIER.

→ Terminals can match lexemes!!!

So we'll define a base class for expressions. Then, for each kind of expression—each production under expression—we create a subclass that has fields for the nonterminals specific to that rule. This way, we get a compile error if we, say, try to access the second operand of a unary expression.

what is
 the essence of each subclass? A name, and a list of typed fields. That's it.

A (Not Very) Pretty Printer

When we debug our parser and interpreter, it's often useful to look at a parsed syntax tree and make sure it has the structure we expect.

Literal expressions are easy—they convert the value to a string with a little check to handle Java's null standing in for Lox's nil. The other expressions have subexpressions, so they use this `parenthesize()` helper method:

lox/AstPrinter.java
 create new file

```
<<
lox/AstPrinter.java
add after print()
<<
```

This recursion is also why people think the Visitor pattern itself has to do with trees.

```
private String parenthesize(String name, Expr... exprs) {
    StringBuilder builder = new StringBuilder();
    builder.append("(").append(name);
    for (Expr expr : exprs) {
        builder.append(" ");
        builder.append(expr.accept(this));
    }
    builder.append(")");
    return builder.toString();
}
```

It takes a name and a list of subexpressions and wraps them all up in parentheses, yielding a string like:
 (+ 1 2)

Note that it calls `accept()` on each subexpression and passes in itself. This is the recursive step that lets us print an entire tree.

Using our handy dandy new notation, here's a grammar for those:

```
expression → literal
            | unary
            | binary
            | grouping;
literal → NUMBER | STRING | "true" | "false" | "nil";
grouping → "(" expression ")";
unary → ( "-" | "!" ) expression;
binary → expression operator expression;
operator → "==" | "!=" | "<" | "<=" | ">" | ">="
          | "+" | "-" | "*" | "/";
```

Working with Trees

Put on your imagination hat for a moment. Even though we aren't there yet, consider what the interpreter will do with the syntax trees. Each kind of expression in Lox behaves differently at runtime. That means the interpreter needs to select a different chunk of code to handle each expression type. With tokens, we can simply switch on the `TokenType`. But we don't have a "type" enum for the syntax trees, just a separate Java class for each one.

We could write a long chain of type tests:

```
if (expr instanceof Expr.Binary) {
    // ...
} else if (expr instanceof Expr.Grouping) {
    // ...
} else // ...
```

But all of those sequential type tests are slow. Expression types whose names are alphabetically later would take longer to execute because they'd fall through more if cases before finding the right type. That's not my idea of an elegant solution.

We have a family of classes and we need to associate a chunk of behavior with each one. The natural solution in an object-oriented language like Java is to put that behavior into methods on the classes themselves. We could add an abstract `interpret()` method on `Expr` which each subclass then implements to interpret itself.

This works alright for tiny projects, but it scales poorly. Like I noted before, these tree classes span a few domains. At the very least, both the parser and interpreter will mess with them. As you'll see later, we need to do name resolution on them. If our language was statically typed, we'd have a type checking pass.

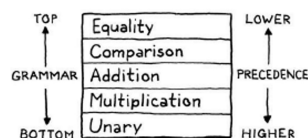
If we added instance methods to the expression classes for every one of those operations, that would smush a bunch of different domains together. That violates separation of concerns and leads to hard to maintain code.

Parsing

The only remaining piece is parsing—transmogrifying a sequence of tokens into one of those syntax trees

Given a string—a series of tokens—we map those tokens to terminals in the grammar to figure out which rules could have generated that string.

Recursive descent is the simplest way to build a parser, and doesn't require using complex parser generator tools like Yacc, Bison or ANTLR. All you need is straightforward hand-written code. Don't be fooled by its simplicity, though.



Recursive descent is considered a top-down parser because it starts from the top or outermost grammar rule (here expression) and works its way down into the nested subexpressions before finally reaching the leaves of the syntax tree. This is in contrast with bottom-up parsers like LR that start with primary expressions and compose them into larger and larger chunks of syntax.

Like the scanner, the parser consumes a flat input sequence, only now we're reading tokens instead of characters. Each method for parsing a grammar rule produces a syntax tree for that rule and returns it to the caller.

SYNTAX TREE

- Unary $\rightarrow$ single operand
- Binary $\rightarrow$ two
- Liternal $\rightarrow$ none

```
package com.craftinginterpreters.lox;
```

Class lives in the declaration grammar rule!!

Like we do, we're gonna start with syntax. A `class` statement introduces a new name, so it lives in the `declaration` grammar rule.

```

declaration → classDecl
            | funDecl
            | varDecl
            | statement ;

classDecl   → "class" IDENTIFIER "{" function* "}" ;

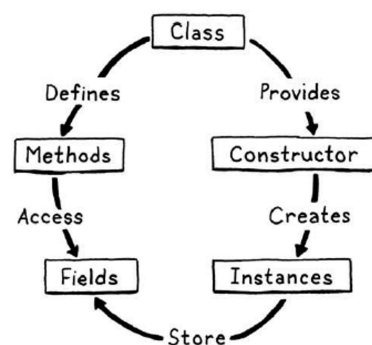
```

The new `classDecl` rule relies on the `function` rule we defined earlier. To refresh your memory:

```
function      → IDENTIFIER "(" parameters? ")" block ;
parameters   → IDENTIFIER ( "," IDENTIFIER ) * ;
```

```
parameters → IDENTIFIER ( "," IDENTIFIER ) * ;
```

I'll need to define this to grow the
 May need to look to scope as well



In plain English, a class declaration is the class keyword, followed by the class's name, then a curly brace body. Inside that body is a list of method declarations.

Yes, **in this current form**, you're setting yourself up for a **circular dependency** problem. Let's break it down simply:

O erro que você está enfrentando:

```
undefined reference to `vtable for BinaryExpression`
```

Yes, in this current form, you're setting yourself up for a **circular dependency** problem. Let's break it down simply:

Where the circular dependency happens

You have:

- Expression.h (defining BinaryExpression, GroupingExpression, etc.)
- IPrettyPrintVisitor.h (defining the visitor interface)
- PrettyPrintVisitor.h (implements the visitor methods using expression classes)

Now, both:

- Expression subclasses include (or need to know about) IPrettyPrintVisitor, and
- IPrettyPrintVisitor needs to know about the concrete expression classes (BinaryExpression, etc.)

This creates a **cycle**:

IPrettyPrintVisitor.h --> needs BinaryExpression (from Expression.h)

Expression.h --> needs IPrettyPrintVisitor.h

How to fix the circular dependency

Use **forward declarations** in IPrettyPrintVisitor.h. This avoids including the full definition of each expression class.

Change this:

```
#include "BinaryExpression.h" // or full Expression headers
```

To this:

```
class BinaryExpression;
class GroupingExpression;
class LiteralExpression;
class UnaryExpression;
```

This tells the compiler:

```
"These are classes that exist; I'll see their full definitions later."
```

Why it works

The visitor interface only uses **pointers** to the expression classes:

```
virtual std::string visitBinaryExpression(BinaryExpression* expr) = 0;
```

Since it's just using pointers (not the full class), a forward declaration is enough.

But in the .cpp file where you implement the visitor methods, **you must include the full class definitions**, like this:

```
#include "BinaryExpression.h"
```

```
#include "LiteralExpression.h"
```

```
// etc.
```

Summary

To avoid circular dependencies:

- In IPrettyPrintVisitor.h: **use forward declarations** of expression classes.
- In .cpp files (like PrettyPrintVisitor.cpp): **include the full headers** of the expressions.
- Avoid including Expression.h inside IPrettyPrintVisitor.h if not absolutely necessary.

Let me know if you want me to rewrite the headers using this correct pattern.

O erro que você está enfrentando:

undefined reference to `vtable for BinaryExpression`

significa que o **compilador encontrou as declarações das classes com métodos virtuais**, mas **não conseguiu encontrar as implementações dos destrutores virtuais**. Como resultado, ele **não conseguiu gerar a vtable (virtual table)**, necessária para o polimorfismo em tempo de execução.

Solução

Você declarou os destrutores, mas **não os implementou** em lugar nenhum. O linker não consegue encontrá-los.

Por exemplo, no seu Expressions.h:

```
class BinaryExpression : public Expression {
public:
    BinaryExpression(Expression* left, Expression* right, const std::string& op);
    ~BinaryExpression(); // <- Você declarou, mas não implementou
    ...
};
```

Você precisa **implementar os destrutores no Expressions.cpp**:

```
#include "Expressions.h"
// Construtores
BinaryExpression::BinaryExpression(Expression* left, Expression* right, const std::string& op)
    : left(left), right(right), op(op) {}
GroupingExpression::GroupingExpression(Expression* expression)
    : expression(expression) {}
LiteralExpression::LiteralExpression(const std::string& value)
    : value(value) {}
UnaryExpression::UnaryExpression(const std::string& op, Expression* right)
    : op(op), right(right) {}
// Destrutores (mesmo que vazios, precisam ser definidos!)
BinaryExpression::~BinaryExpression() {}
GroupingExpression::~GroupingExpression() {}
LiteralExpression::~LiteralExpression() {}
UnaryExpression::~UnaryExpression() {}
```

Por que isso acontece?

Toda classe com métodos virtuais (como accept) gera uma **vtable** e, para isso, **precisa que o destrutor virtual tenha implementação real** (mesmo que vazia).

Se o destrutor não for implementado no .cpp, o linker reclamará com esse erro porque não consegue montar a vtable da classe.

Checklist para resolver

1. **Verifique se há métodos virtuais declarados (como accept)**
2. **Garanta que todos os destrutores virtuais tenham implementação (mesmo vazia!)**
3. **Coloque essas implementações no .cpp, não no .h**

Depois disso, seu projeto deve compilar com sucesso e você poderá seguir com a escrita dos testes unitários, que posso te mostrar em seguida com base no seu Visitor PrettyPrintVisitor.

Quer que eu te mostre um exemplo de teste usando Google Test com uma LiteralExpression e o PrettyPrintVisitor?